

学号： 2106410124



沈阳建筑大学

智能手机应用程序开发 课后作业

2023 年_秋_季学期

姓 名： 纪天一 专业班级： 计算机 2101 班
指导教师： 郑新录

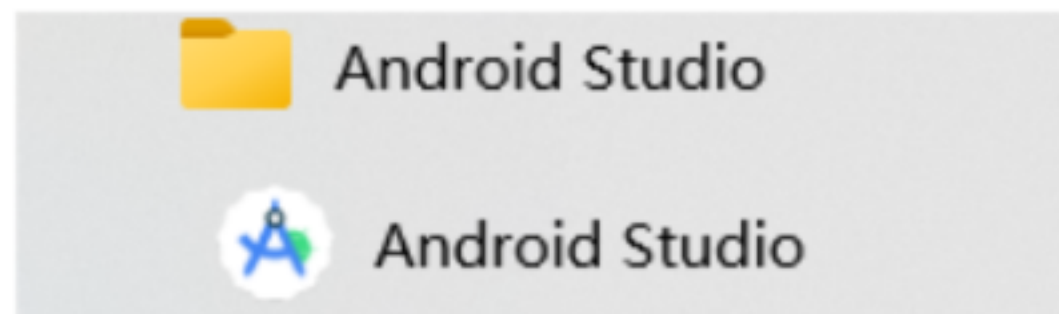
第一章作业：

P22 1. 下载最新版本的 Android Studio，安装后如果需要更新 Gradle，则将其更新到最新版本。

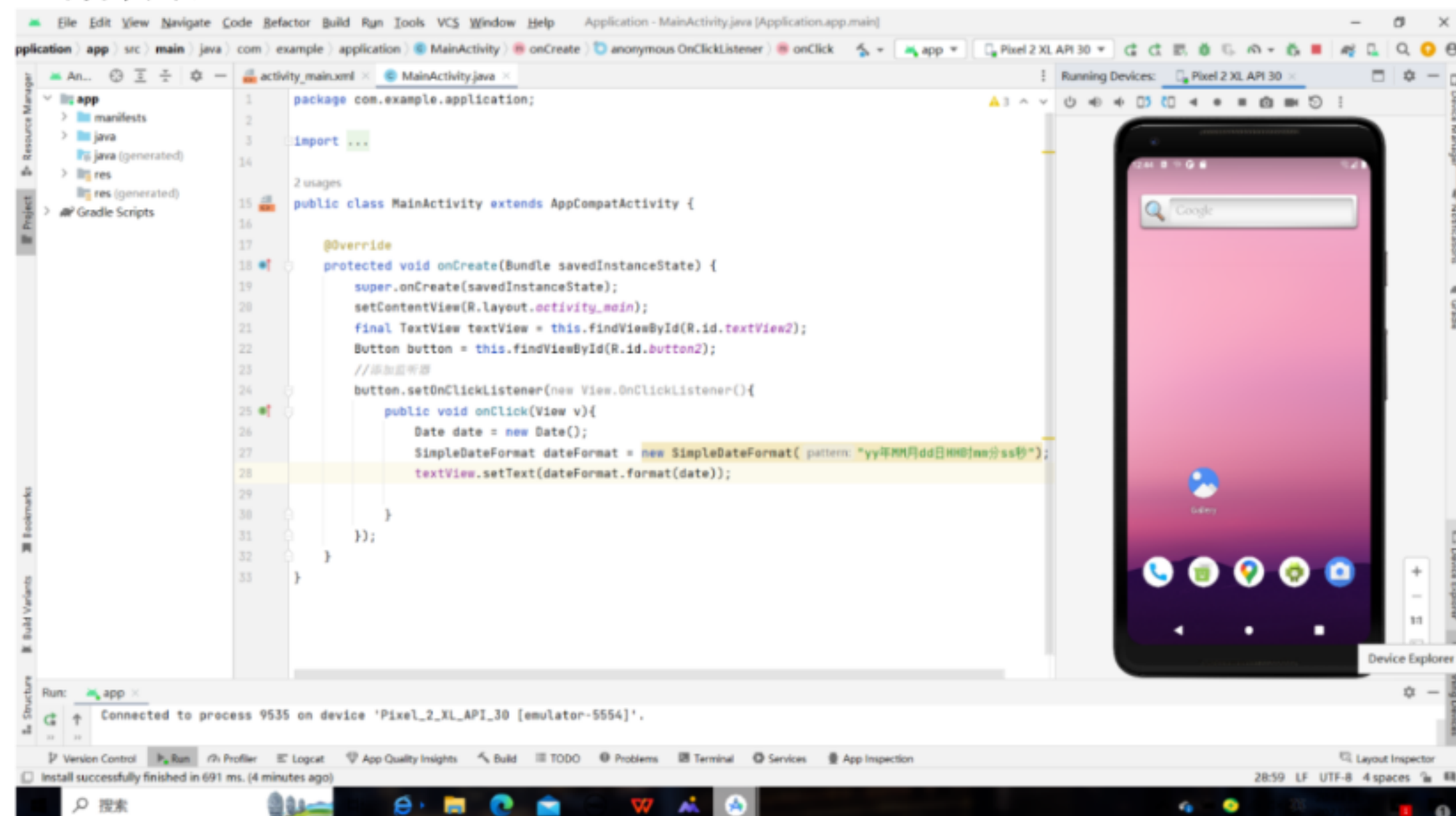
P22 2. 创建一个任意类型的手机 App 工程，分别使用虚拟设备和物理设备运行。

已安装软件：

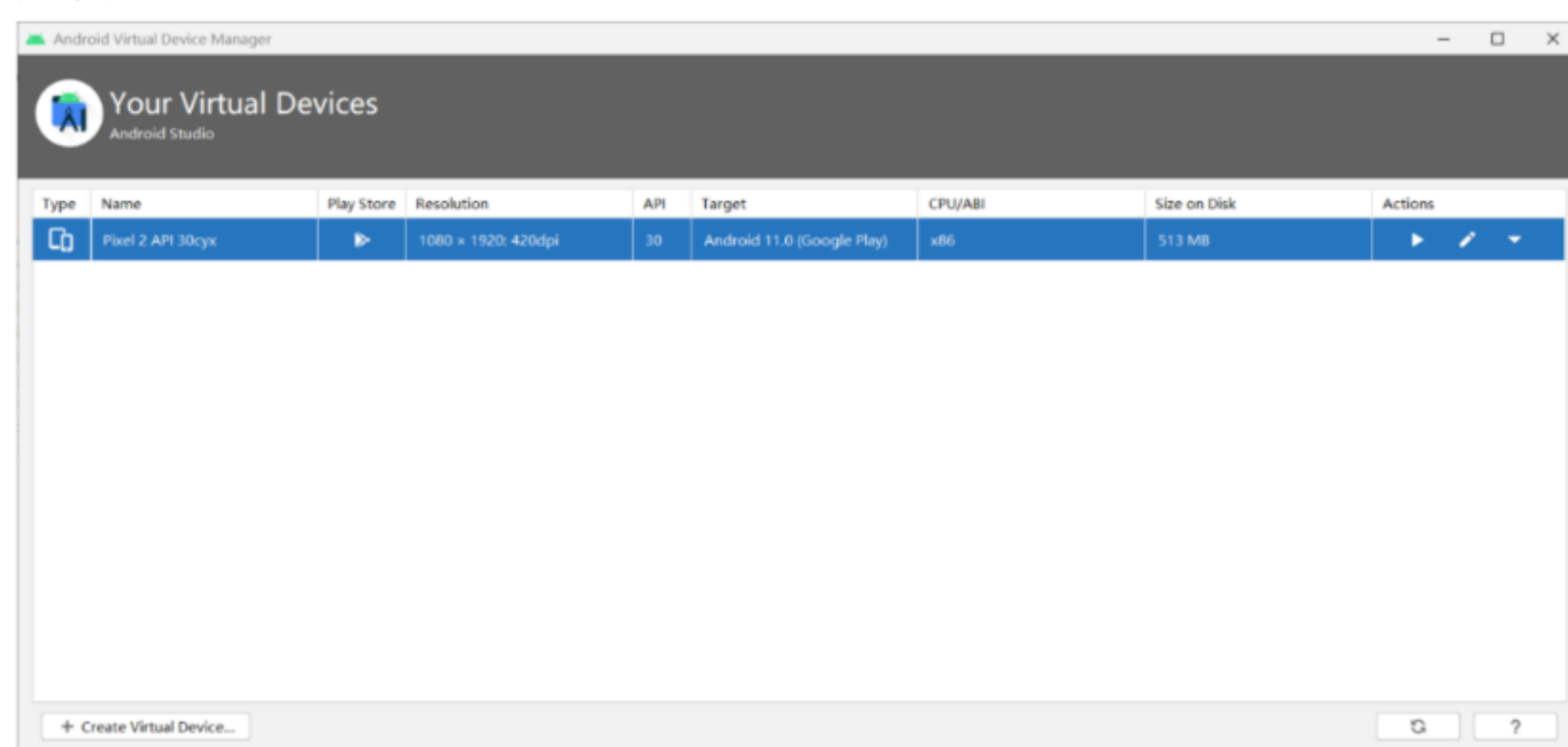
Android Studio 已是最新版本，Gradle 已更新到最新版本。



运行界面：



安装虚拟机：



虚拟机运行：



第二章作业：

P73 2.2 实现多选题的界面效果。

核心代码：

```
public class MainActivity extends AppCompatActivity {
    private CheckBox checkBoxJava;
    private CheckBox checkBoxPython;
    private CheckBox checkBoxC;
    private Button submitButton;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        checkBoxJava = findViewById(R.id.checkbox_java);
        checkBoxPython = findViewById(R.id.checkbox_python);
        checkBoxC = findViewById(R.id.checkbox_c);
        submitButton = findViewById(R.id.submit_button);
        submitButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String selectedOptions = "";
                if (checkBoxJava.isChecked()) {
                    selectedOptions += "Java ";
                }

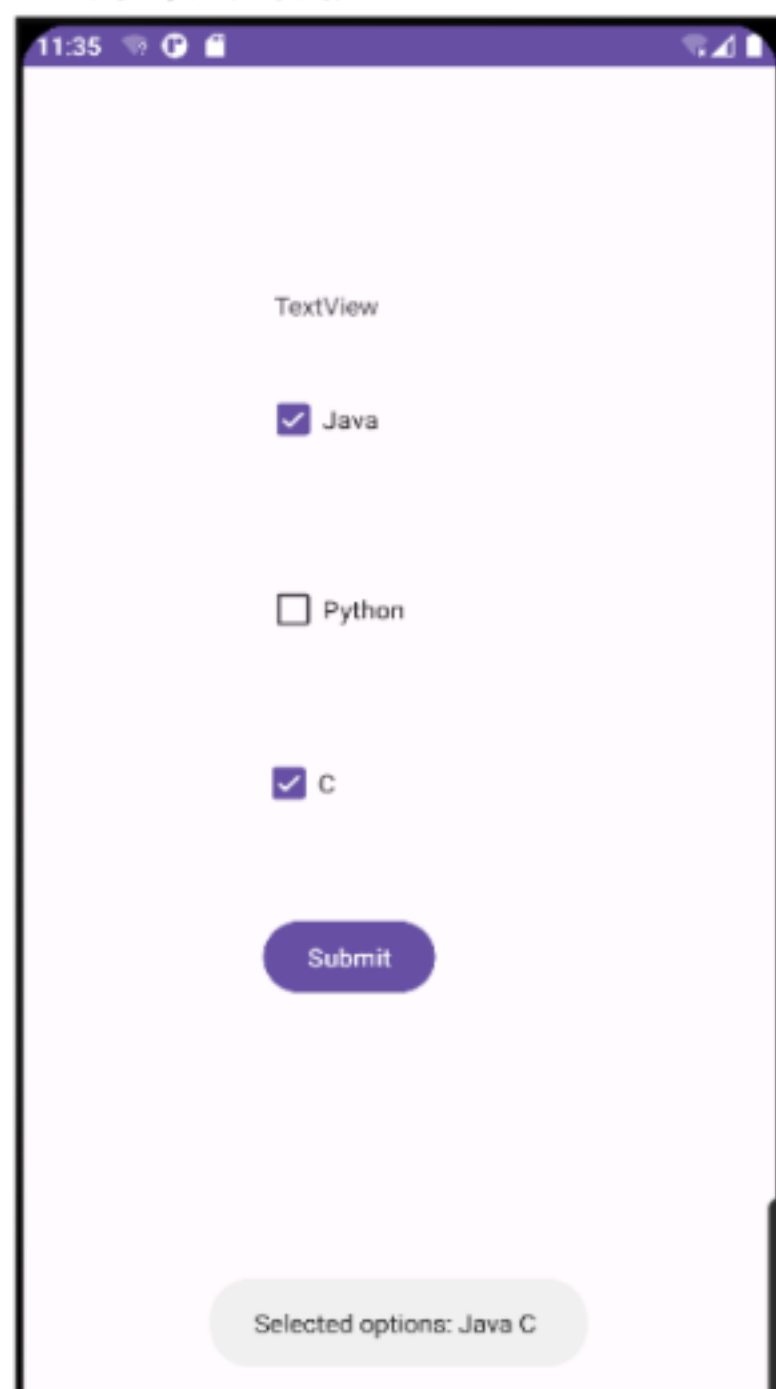
                if (checkBoxPython.isChecked()) {
```

```

        selectedOptions += "Python ";
    }
    if (checkBoxC.isChecked()) {
        selectedOptions += "C ";
    }
    Toast.makeText(MainActivity.this, "Selected options: " +
selectedOptions, Toast.LENGTH_SHORT).show();}
    });
}
}

```

运行结果截图：



p73 2.3 实现选择出生日期的界面效果。

核心代码：

```

public class MainActivity extends AppCompatActivity {
    private Button btnSelectDate;
    private TextView tvSelectedDate;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnSelectDate = findViewById(R.id.button);
        tvSelectedDate = findViewById(R.id.textView);
        btnSelectDate.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                showDatePickerDialog();
            }
        });
    }
}

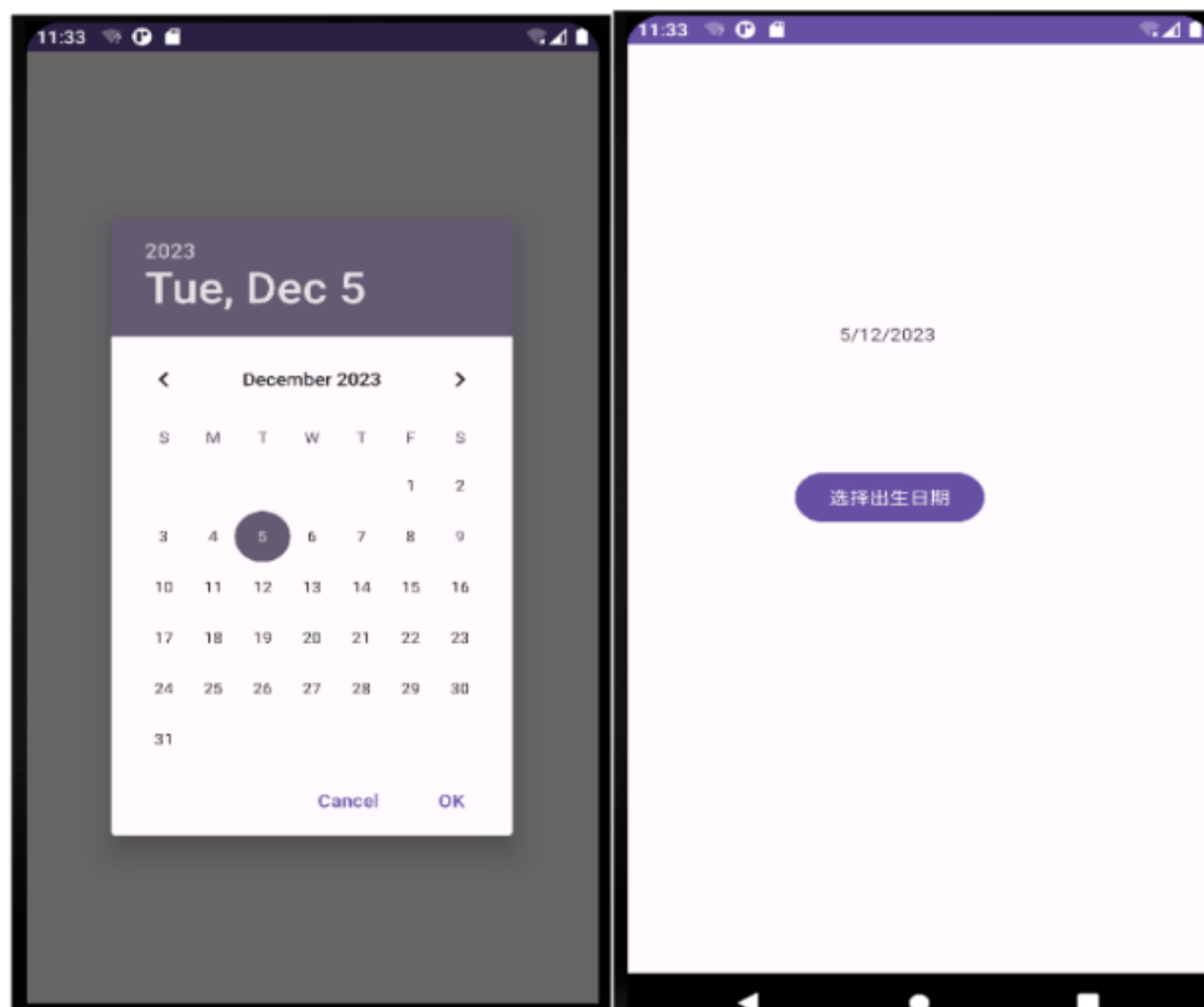
```

```

    });
}
private void showDatePickerDialog() {
    Calendar calendar = Calendar.getInstance();
    int year = calendar.get(Calendar.YEAR);
    int month = calendar.get(Calendar.MONTH);
    int day = calendar.get(Calendar.DAY_OF_MONTH);
    DatePickerDialog datePickerDialog = new
DatePickerDialog(MainActivity.this,
    new DatePickerDialog.OnDateSetListener() {
        @Override
        public void onDateSet(DatePicker view, int year, int
month, int dayOfMonth) {
            String selectedDate = dayOfMonth + "/" + (month
+ 1) + "/" + year;
            tvSelectedDate.setText(selectedDate);
        }
    }, year, month, day);
    datePickerDialog.show(); }
}

```

运行结果截图：



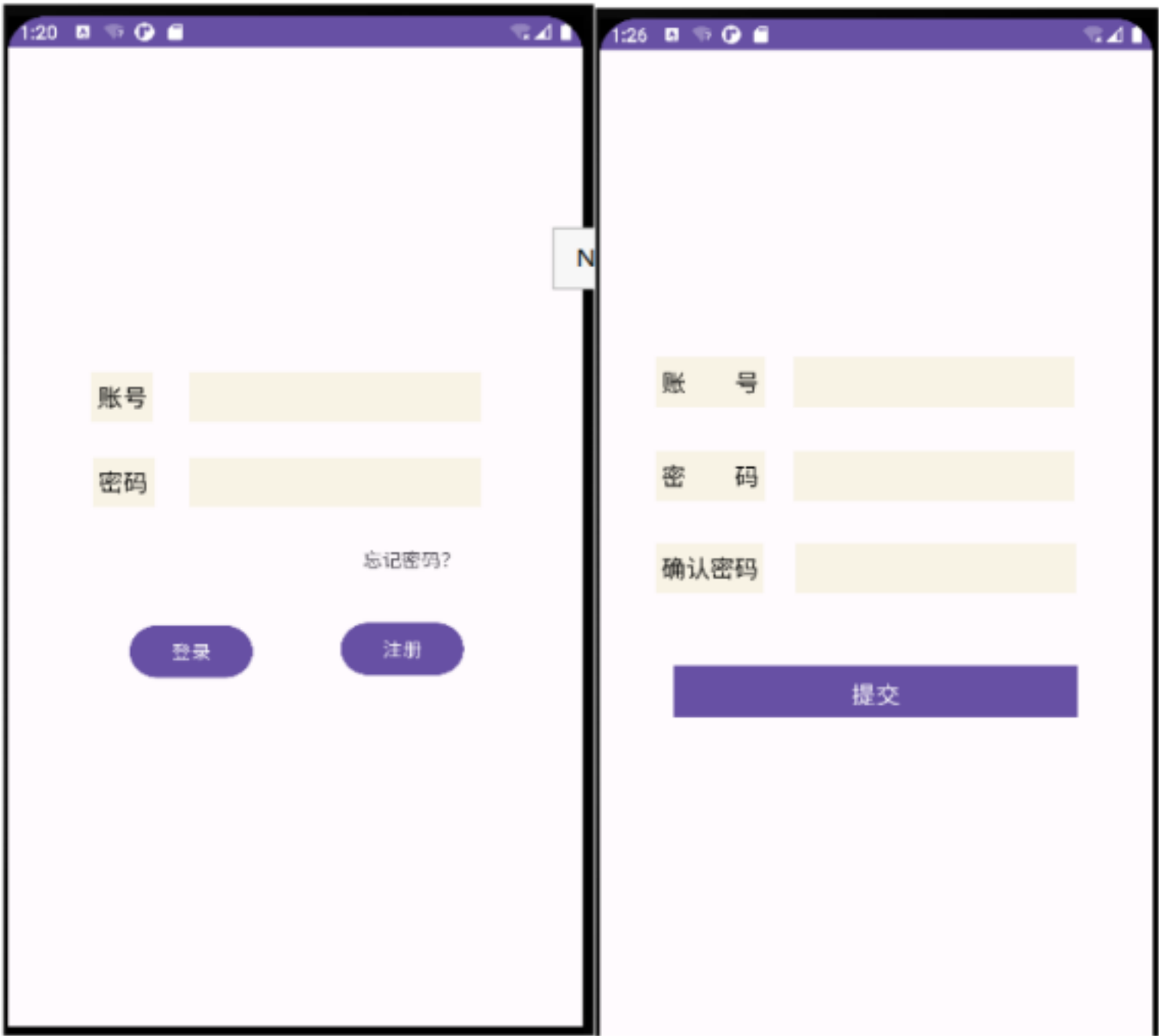
第三章：

p87 3.1 实现注册界面的布局效果，至少包括用户名、密码、忘记密码、登录等控件。

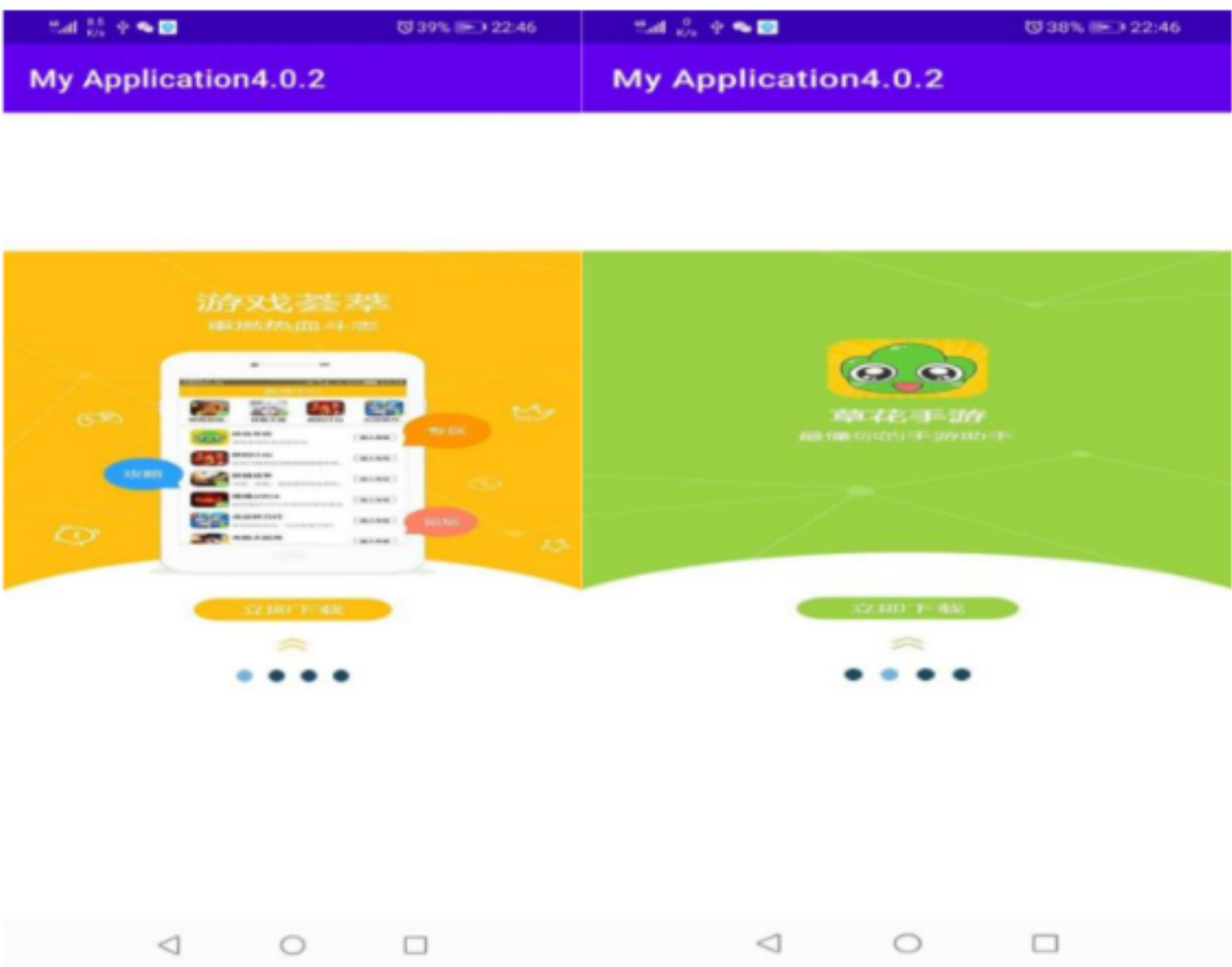
核心代码：

使用控件直接生成。

运行结果截图：
登录界面和注册界面：



第四章：
p122 4.3 实现京东首页的轮播广告效果，至少包括 3 个产品广告的图像。
使用 UI 控件。
运行结果截图：





第五章：

p150 5.1 实现登录后显示用户名的效果。输入用户名、密码后登录，启动一个新的 Activity 显示用户名。

核心代码：

MainActivity 代码

```
public class MainActivity extends AppCompatActivity {
    private EditText usernameEditText;
    private EditText passwordEditText;
    private Button loginButton;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        usernameEditText = findViewById(R.id.usernameEditText);
        passwordEditText = findViewById(R.id.passwordEditText);
        loginButton = findViewById(R.id.loginButton);
        loginButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String username = usernameEditText.getText().toString();
                String password = passwordEditText.getText().toString();
                // 进行用户名和密码的验证
                if (username.equals("dell") &&
password.equals("123456")) {
                    Intent intent = new Intent(MainActivity.this,
DisplayUsernameActivity.class);
                    intent.putExtra("username", username);
```

```

        startActivity(intent);
    } else {
        Toast.makeText(MainActivity.this, "Invalid username
or password", Toast.LENGTH_SHORT).show();}
    }
});
}
}

```

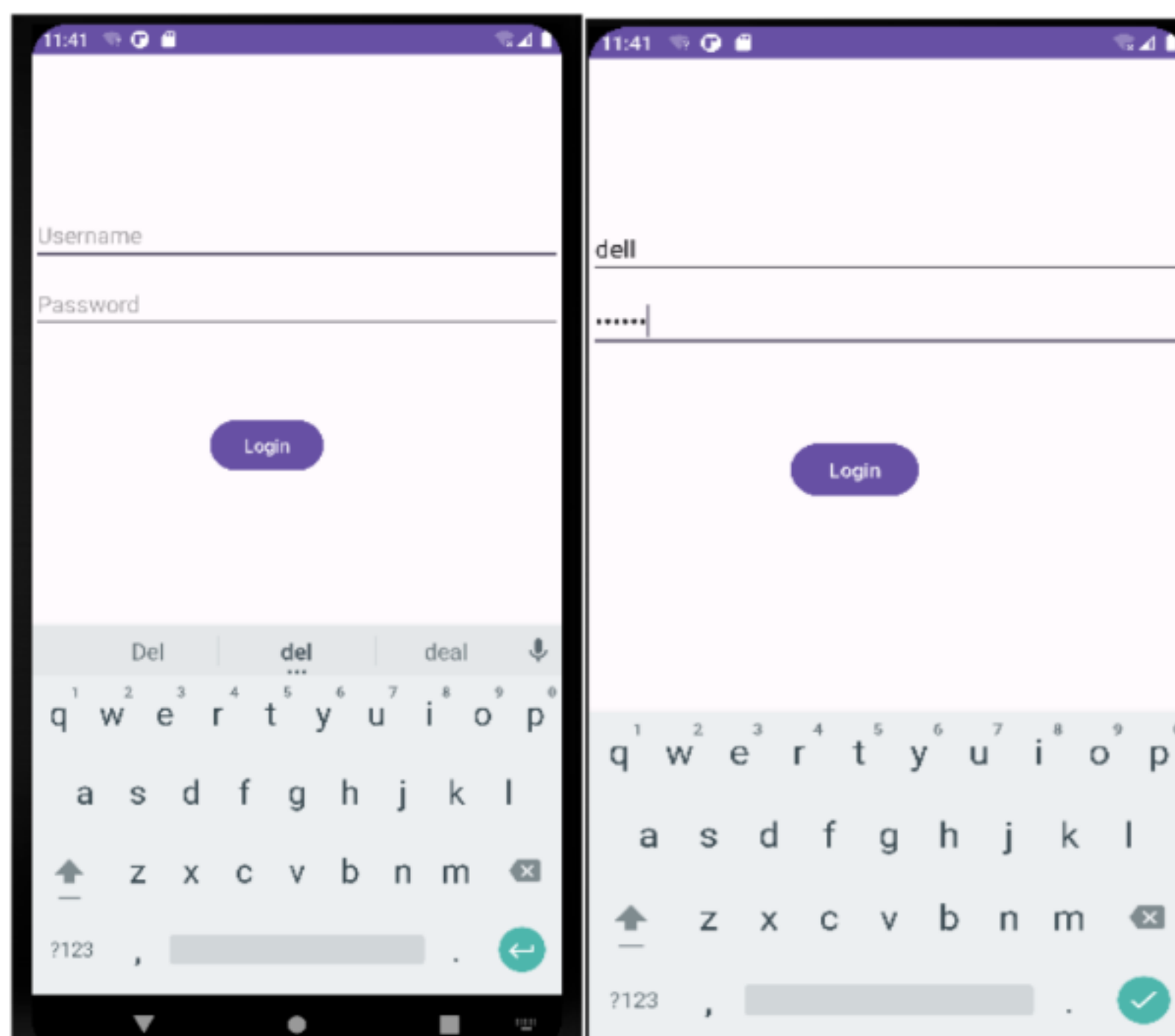
一个新的 Activity 显示用户名：

```

public class DisplayUsernameActivity extends AppCompatActivity {
    private TextView usernameTextView;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_display_username);
        usernameTextView = findViewById(R.id.textview);
        // 获取传递的用户名
        String username = getIntent().getStringExtra("username");
        // 显示用户名
        usernameTextView.setText("Welcome, " + username + "(刘嘉俊!)");
    }
}

```

运行结果截图：



登录成功跳转新的界面：



第六章：

p176 6.3 通过广播接收器实现两个 App 之间通过自定义广播进行数据通信。
结合 C0605 和 C0606 两个例子。

核心代码：

```
findViewById(R.id.button_send_broadcast).setOnClickListener(new  
View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        Intent intent=new Intent();  
        intent.setAction("MyBroadcastReceiver.Custom");  
        intent.putExtra("info","第一条广播");  
        intent.putExtra("info1","1");  
        intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);  
        sendBroadcast(intent);  
    }  
});
```

其中 MyBroadcastReceiver.Custom 是标识的字符串，需要在 605 的 manifests 文件夹中修改<intent-filter>

```
<action android:name="MyBroadcastReceiver.Custom" />  
</intent-filter>
```

进行对应，要保证两者一致才能收到广播，而且 605 中还有一个判断语句

```
if ("MyBroadcastReceiver.Custom".equals(intent.getAction())) {  
    Log.d(TAG, "自定义广播的 info: " + intent.getStringExtra("info"));
```

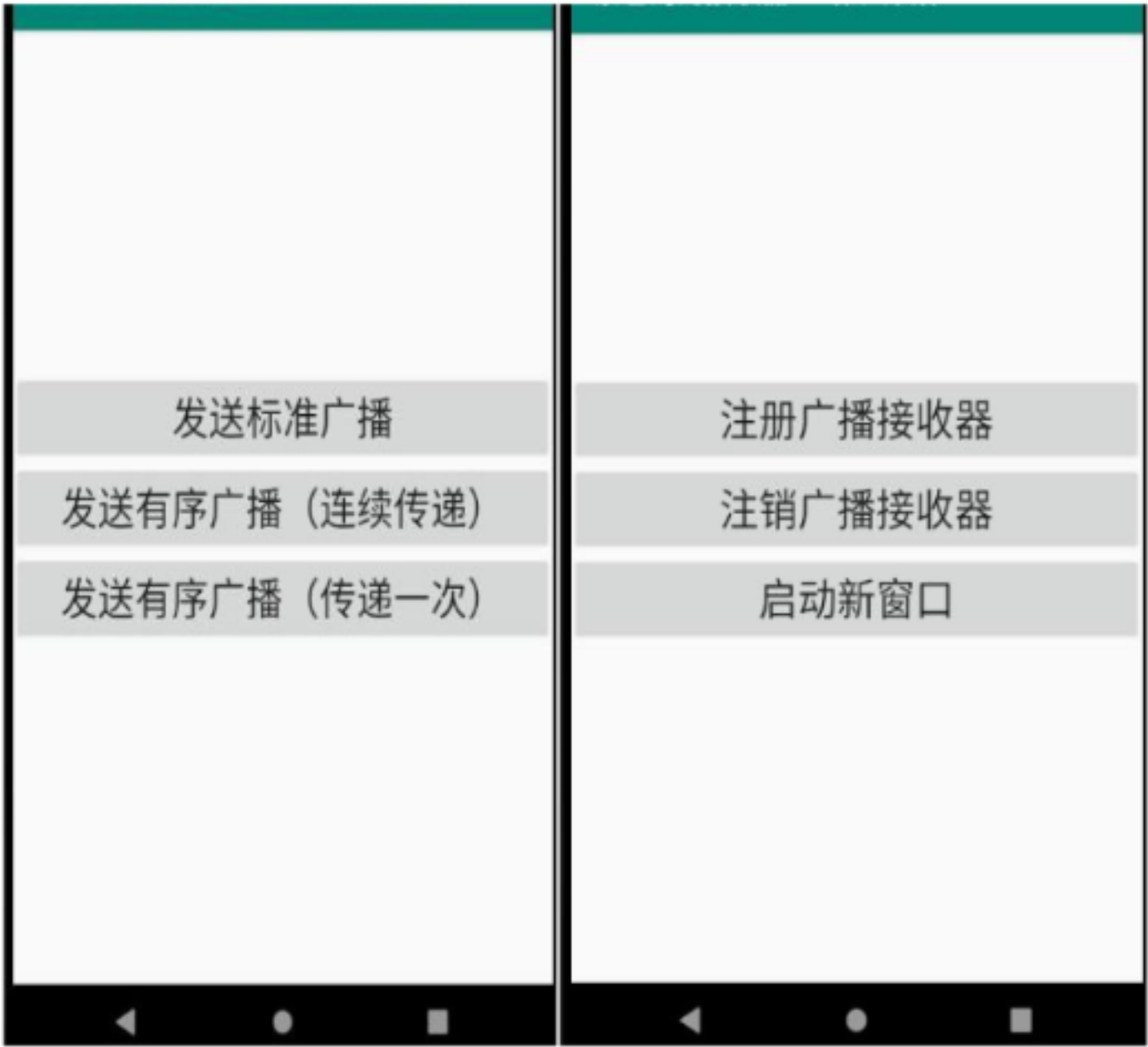
```
    Toast.makeText(context,intent.getStringExtra("info"),Toast.LENGTH_SHORT  
).show();  
}
```

则要保证这三者的字符串一致才能实现两个 APP 之间的通信。由于后台 APP 之间的通信我们看不到，所以这边写了两个语句

```
Log.d(TAG, "自定义广播的 info: " + intent.getStringExtra("info"));
Toast.makeText(context,intent.getStringExtra("info"),Toast.LENGTH_SHORT
).show();
```

第一个语句是在电脑端显示我们收到的广播，第二条语句是在 APP 中显示接收到的广播。如下图。其中 info 是标识广播的身份，如果 606 同时发送了多条广播，每一条广播都有它自己的身份标识，例如 info、info1，info2 等等，intent.getStringExtra("info");里面的 info 就是你要显示哪一条广播。

运行结果截图：
广播发送界面和广播接收界面：



注册广播接收器成功：



发送广播并被广播接收器接收：



第七章：

p213 7.2 使用 SQLite 实现记录登录时间的功能，并且能够查询登录的历史记录。

核心代码：

```
public class ContactSQLiteOpenHelper extends SQLiteOpenHelper {
    static String name="user.db";
    static int dbVersion=1;
    public ContactSQLiteOpenHelper(Context context){
        super(context, name, null, dbVersion);
    }
    @Override
    public void onCreate(SQLiteDatabase db) {
        String sql="create table user(id integer primary key
autoincrement,username varchar(20),password varchar(20),time
varchar(30))";
        db.execSQL(sql);
    }
}
```

同时设置了一个 DBUtils.java 类，专门来调用 ContactSQLiteOpenHelper 生成数据库，以及储存一些对数据库操作的方法。

里面有核心方法例如登录、注册、更新数据等方法。如下面代码中的 login 方法，是判断数据库中是否有与参数 username、password 一样的数据条。可以用于登录时的验证，同时也用于注册时检测是否以及存在用户名。

```
public class DBUtils {
    private ContactSQLiteOpenHelper dbHelper;
    public DBUtils(Context context){
        dbHelper=new ContactSQLiteOpenHelper(context);
    }
    public boolean login(String username,String password){
        SQLiteDatabase sdb=dbHelper.getReadableDatabase();
        String sql="select * from user where username=? and password=?";
        Cursor cursor=sdb.rawQuery(sql, new
String[]{username,password});
        if(cursor.moveToFirst()==true){
            cursor.close();
            return true;
        }
        return false;
    }
    public boolean register(String user,String Pass,String time){
        SQLiteDatabase sdb=dbHelper.getReadableDatabase();
        String sql="insert into user(username,password,time) values(?,?,?)";
        Object obj[]={user,Pass,time};
        sdb.execSQL(sql, obj);
        return true;
    }
    public boolean updatetime(String user,String Pass,String time){
```

```

        SQLiteDatabase sdb=dbHelper.getReadableDatabase();
        ContentValues values = new ContentValues();
        values.put("time",time);
        sdb.update("user",values,"username=?",new String[]{user});
        return true;
    }
}

```

在登录界面点击登录，运行以下代码：

```

public void onClick(View v) {
    user=findViewById(R.id.et_user_name);
    pass=findViewById(R.id.et_psw);
    pass_repeat=findViewById(R.id.et_psw_again);
    String name=user.getText().toString().trim();
    String psw =pass.getText().toString().trim();
    String psw_repeat=pass_repeat.getText().toString();
    SimpleDateFormat formatter = new
SimpleDateFormat("yyyy 年 MM 月 dd 日 HH:mm:ss ");
    Date curDate = new Date(System.currentTimeMillis());
    String str = formatter.format(curDate);
    boolean flag=uService.login(name,psw);
    if(!psw_repeat.equals(psw)){
        Toast.makeText(RegisterActivity.this,
            " 两 次 密 码 输 入 错 误 ",
Toast.LENGTH_SHORT).show();
    }
    else if(flag){
        Toast.makeText(RegisterActivity.this, "注册失败,用户
名已存在", Toast.LENGTH_SHORT).show();
    }
    else {
        Log.i("TAG", name + "_" + psw);
        uService.register(name,psw,str);
        Toast.makeText(RegisterActivity.this, " 注 册 成 功 ",
Toast.LENGTH_LONG).show();
        Intent intent =new
Intent(RegisterActivity.this,MainActivity.class);
        startActivity(intent);
    }
}
}

```

在注册时，在数据库中加入属于自己用户名的数据，分别是用户名、密码、时间，所以说，每个账号的第一次登录时间是它的注册时间。

```
SimpleDateFormat formatter = new SimpleDateFormat("yyyy 年 MM 月 dd
日 HH:mm:ss ");
```

```
Date curDate = new Date(System.currentTimeMillis());
```

```
String str = formatter.format(curDate);
```

这三句代码实现了获取当前系统时间，并转化成字符串 `str`，调用 `register(name,psw,str)`实现数据的插入。

之后的业务逻辑是，每次登录不再插入数据，而是更新以用户名为验证，每次登录都更新 `time`。

登录界面也是如此，在验证通过之后，获取当前系统时间并转化成 `str` 字符串，调用相关方法 `uService.updatetime(strUsername,strPassword,str)`去更新数据。并传入用户名、密码、登录时间信息。

```
intent.putExtra("Username",strUsername);
```

```
intent.putExtra("Password",strPassword);
```

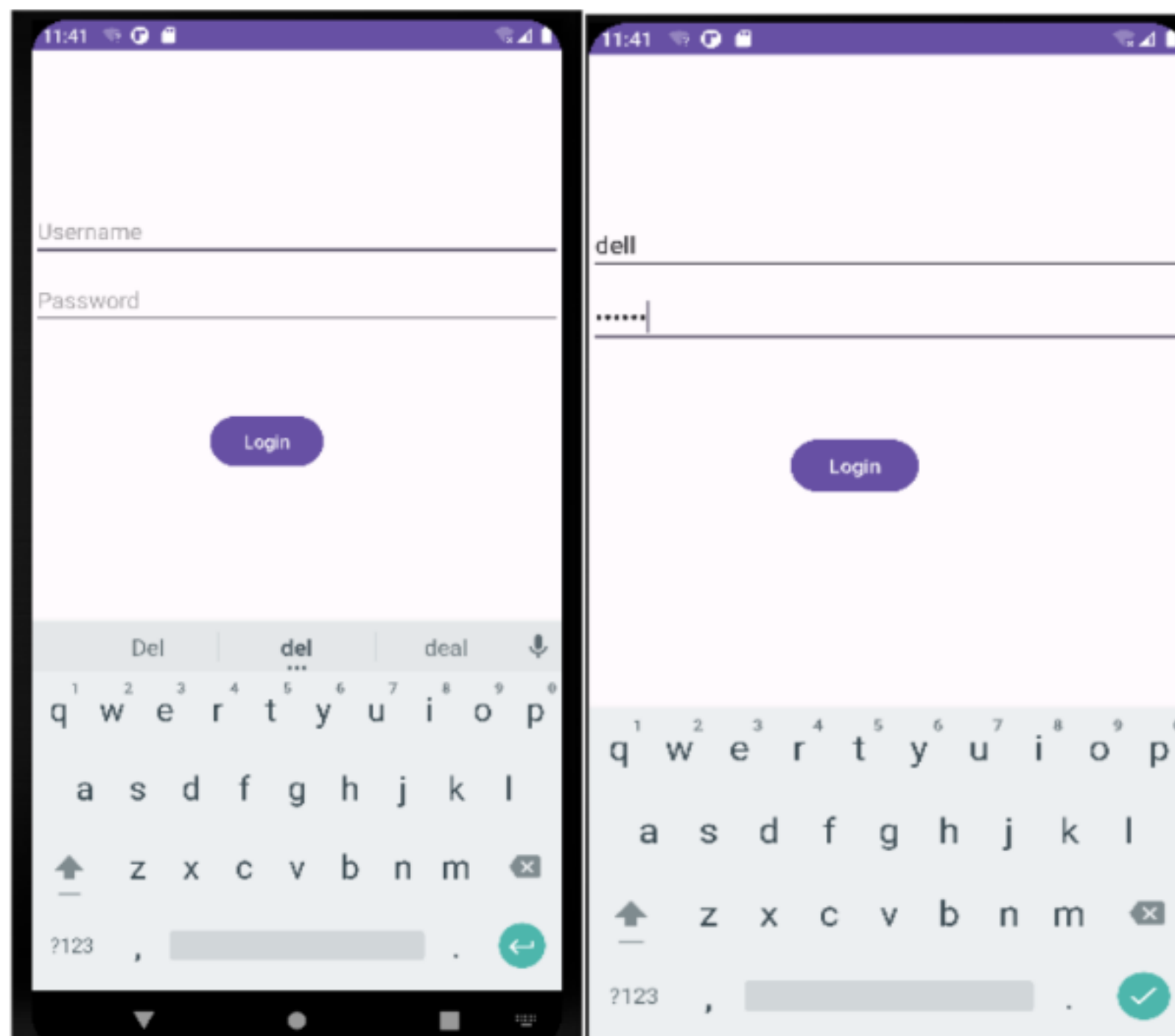
```
intent.putExtra("time",str);
```

登录后的主页接收信息

```
String username = intent.getStringExtra("Username");
```

```
String password = intent.getStringExtra("Password");
```

```
String time=intent.getStringExtra("time");
```





登录成功显示登录用户名和密码，以及最后一次登录时间。

第八章：

p271 8.1 使用 Camera2 实现定时拍照的功能。

结合 C0805 的例子。

核心代码：

实现倒计时功能代码：

TextView tv = findViewById(R.id.show_time);

CountDownTimer cdt = new CountDownTimer(5000, 1000)// 参数

1：计时总时间，参数 2：每次扣除时间数

```
{
    int timer = 5;
    @Override
    public void onTick(long millisUntilFinished)
    {
        tv.setText(timer + "S");
        timer--;
        //定时时间到执行动作;
    }

    @Override
    public void onFinish() {
        tv.setText("");
        takePhoto();
    }
};
```

```
cdt.start();
```

拍照功能代码基于例子 C0805 实现。

运行结果截图：

